Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Санкт-Петербургский национальный исследовательский университет информационных технологий, механики и оптики»

Факультет «Программной Инженерии и Компьютерной Техники»

Дисциплина «Системное программное обеспечение»

Отчёт к практическому заданию № 2

Вариант 2

Входной язык 2

Выполнил:

Студент группы P4114 Тягельский А. А.

Проверил:

Доцент факультета ПИиКТ Кореньков Юрий Дмитриевич

Санкт-Петербург, 2025

Цель работы:

Реализовать построение графа потока управления посредством анализа дерева разбора для набора входных файлов. Выполнить анализ собранной информации и сформировать набор файлов с графическим представлением для результатов анализа.

Задачи:

1. Описать структуры данных, необходимые для представления информации о наборе файлов, наборе подпрограмм и графе потока управления, где:
    a. Для каждой подпрограммы: имя и информация о сигнатуре, граф потока управления, имя исходного файла с текстом подпрограммы.
    b. Для каждого узла в графе потока управления, представляющего собой базовый блок алгоритма подпрограммы: целевые узлы для безусловного и условного перехода (по мере необходимости), дерево операций, ассоциированных с данным местом в алгоритме, представленном в исходном тексте подпрограммы.
2. Реализовать модуль, формирующий граф потока управления на основе синтаксической структуры текста подпрограмм для входных файлов.
    a. Программный интерфейс модуля принимает на вход коллекцию, описывающую набор анализируемых файлов, для каждого файла – имя и соответствующее дерево разбора в виде структуры данных, являющейся результатом работы модуля, созданного по заданию 1 (п. 3.b).
    b. Результатом работы модуля является структура данных, разработанная в п. 1, содержащая информацию о проанализированных подпрограммах и коллекция с информацией об ошибках.
    c. Посредством обхода дерева разбора подпрограммы, сформировать для неё граф потока управления, порождая его узлы и формируя между ними дуги в зависимости от синтаксической конструкции, представленной данным узлом дерева разбора: выражение, ветвление, цикл, прерывание

цикла, выход из подпрограммы – для всех синтаксических конструкций по варианту (п. 2.b).

d. С каждым узлом графа потока управления связать дерево операций, в котором каждая операция в составе текста программы представлена как совокупность вида операции и соответствующих операндов (см задание 1, пп. 2.d-g).
e. При возникновении логической ошибки в синтаксической структуре при обходе дерева разбора, сохранить в коллекции информацию об ошибке и её положении в исходном тексте.

3. Реализовать тестовую программу для демонстрации работоспособности созданного модуля.
   a. Через аргументы командной строки программа должна принимать набор имён входных файлов, имя выходной директории.
   b. Использовать модуль, разработанный в задании 1 для синтаксического анализа каждого входного файла и формирования набора деревьев разбора.
   c. Использовать модуль, разработанный в п. 2 для формирования графов потока управления каждой подпрограммы, выявленной в синтаксической структуре текстов, содержащихся во входных файлах.
   d. Для каждой обнаруженной подпрограммы вывести представление графа потока управления в отдельный файл с именем “sourceName.functionName.ext” в выходной директории, поумолчанию размещать выходной файлы в той же директории, что соответствующий входной.
   e. Для деревьев операций в графах потока управления всей совокупности подпрограмм сформировать граф вызовов, описывающий отношения между ними в плане обращения их друг к другу по именам и вывести его представление в дополнительный файл, по-умолчанию размещаемый рядом с файлом, содержащим подпрограмму main.
   f. Сообщения об ошибке должны выводиться тестовой программной (не модулем, отвечающим за анализ!) в стандартный поток вывода ошибок

Ход работы

1 Описание структуры данных для представления информации о наборе файлов, наборе подпрограмм и графе потока управления

Итоговая структура данных, которая получается при анализе дерева разбора и составлении графа потока управления, называется SubprogramCollection. Структура данных содержит набор SubprogramInfo, каждый из элементов SubprogramInfo хранит наименование файла, наименование метода, наименование и тип параметров метода, возвращаемый тип метода, локальные переменные и их типы, флаг наличия тела метода, а также граф потока управления. Граф потока управления (структура ControlFlowGraph) состоит из входного узла, выходного узла и набора других узлов. При этом каждая нода имеет id, тип (базовая нода, if нода и т.д.), набор операций, ссылку на следующую ноду при выполнении условия и дефолтная ссылка на следующую ноду. Помимо этого, представлена структура для хранения обращений методов друг к другу – CallGraph. Код для представления структуры данных и объявленные методы находятся в файле cfg_builder_module.h и представлен ниже:

```
// Новые структуры для CFG
typedef enum {
   NODE_BASIC_BLOCK,
   NODE_ENTRY,
   NODE_EXIT,
   NODE_IF,
   NODE_WHILE,
   NODE_REPEAT_CONDITION,
   NODE_BREAK,
   NODE_RETURN
} NodeType;


typedef struct CFGNode {
   int id;
   NodeType type;
   // Для basic block
```

```
    OpNode** statements;
    int stmt_count;
    struct CFGNode* nextDefault;
    struct CFGNode* nextConditional;
} CFGNode;


// Типы ребер CFG
typedef enum {
    EDGE_CLASSIC,   // Обычное ребро потока управления
    EDGE_TRUE,      // Ребро для true ветви условия
    EDGE_FALSE,     // Ребро для false ветви условия
    EDGE_BREAK,     // Ребро для break
    EDGE_CONTINUE   // Ребро для continue (если будет)
} EdgeType;

// Структура для хранения ребра с меткой
typedef struct CFGEdge {
    CFGNode* from;
    CFGNode* to;
    EdgeType type;
} CFGEdge;

typedef struct {
    CFGNode** exits;
    CFGEdge** exitsEdges;
    int exit_count;
} FlowResult;

typedef struct ControlFlowGraph {
CFGNode* entry;
CFGNode* exit;
CFGNode** nodes;
int node_count;
int max_nodes;
CFGEdge** edges;
int edge_count;
```

```
int max_edges;
} ControlFlowGraph;


typedef struct {
char* name;
char** param_names;
char** param_types;
int param_count;
char* return_type;
char** local_names;
char** local_types;
int local_count;
char* source_file;
ControlFlowGraph* cfg;
bool has_body;
} SubprogramInfo;

typedef struct {
    SubprogramInfo* items;
    int count;
} SubprogramCollection;

typedef struct {
    char* caller_name;
    char* callee_name;
} CallGraphEdge;

typedef struct {
    char** node_names;
    int node_count;
    CallGraphEdge* edges;
    int edge_count;
} CallGraph;


ControlFlowGraph* buildCFG(pANTLR3_BASE_TREE block_node);
```

```
SubprogramInfo* generateSubprogramInfo(const char* source_file,
pANTLR3_BASE_TREE tree);
SubprogramCollection generateSubprogramInfoCollection(const char*
source_file, pANTLR3_BASE_TREE tree);
void cfgToDot(ControlFlowGraph* cfg, FILE* out);
void cfgNodesToDot(ControlFlowGraph* cfg, FILE* out);
void freeCFG(ControlFlowGraph* cfg);

// Subprogram helpers
void freeSubprogramCollection(SubprogramCollection* collection);

// Call graph helpers
CallGraph* buildCallGraph(const SubprogramCollection* collection);
void callGraphToDot(const CallGraph* graph, FILE* out);
void freeCallGraph(CallGraph* graph);
```

2 Реализация модуля, формирующего граф потока управления на основе синтаксической структуры текста подпрограмм для входных файлов.

Для взаимодействия с программой и передачи нескольких входных файлов с текстом программы разработан новый API. Команды для запуска созданной утилиты:

```
Base usage:
MyCompiler.exe <input_file> <output_ast_dir> <output_cfg_dir>
Multiple files processing mode:
S:\CLionProjects\MyCompiler\cmake-build-debug\MyCompiler.exe --
multiple <input_file1> <input_file2> ... <input_fileN>
Options:
--help        Display this help message
--multiple    Enter multiple files mode
```

Для построения графа потока управления конкретного метода используется функция ControlFlowGraph*

buildCFG(pANTLR3_BASE_TREE block_node); для генерации структуры данных, хранящей полную информацию о методе, используется функция SubprogramInfo* generateSubprogramInfo(const char* source_file, pANTLR3_BASE_TREE tree); для генерации итоговой структуры данных с набором SubprogramInfo используется функция SubprogramCollection generateSubprogramInfoCollection(const char* source_file, pANTLR3_BASE_TREE tree); для построения графа вызовов используется отдельная функция CallGraph* buildCallGraph(const SubprogramCollection* collection), принимающая на вход итоговую структуру данных с набором SubprogramInfo; также присутствуют вспомогательные функции для отображения графа вызовов, графа потока управления, освобождения ресурсов и тп.

Код созданной утилиты находится в файле cfg_builder_module.c, ниже приведены несколько основополагающих методов созданного класса:

```
static FlowResult processStatement(pANTLR3_BASE_TREE node,
                        ControlFlowGraph* cfg,
                        FlowResult flow_result)
{
   const char* node_text = get_ast_node_text(node);

   // Обработка узла IF.
   if (strcmp(node_text, "IF") == 0)
   {
      FlowResult new_flow_result = processIfStatement(node, cfg, flow_result);

      return new_flow_result;
   }
   // Обработка узла while
   else if (strcmp(node_text, "WHILE") == 0)
   {
      FlowResult new_flow_result = processWhileStatement(node, cfg, flow_result);

      return new_flow_result;
   }
   else if (strcmp(node_text, "REPEAT") == 0)
   {
      // Обработка цикла repeat
      FlowResult new_flow_result = processRepeatStatement(node, cfg, flow_result);

      return new_flow_result;
```

```
    }
    else if (strcmp(node_text, "BREAK") == 0)
    {
        // Обработка break
        FlowResult new_flow_result = processBreakStatement(node, cfg, flow_result);

        return new_flow_result;
    }
    else if (strcmp(node_text, "BLOCK") == 0)
    {
        // Обработка блока statements
        ANTLR3_UINT32 child_count = node->getChildCount(node);

        FlowResult new_flow_result = flow_result;

        for (ANTLR3_UINT32 i = 0; i < child_count; i++)
        {
            pANTLR3_BASE_TREE child = node->getChild(node, i);
            new_flow_result = processStatement(child, cfg, new_flow_result);
        }

        return new_flow_result;
    }
    else if (strcmp(node_text, "ASSIGN") == 0 || strcmp(node_text, "EXPRESSION") == 0)
    {
        CFGNode* current_block;
        bool continue_current_block = true;
        // Обработка блока statements
        if (flow_result.exit_count == 1 && flow_result.exits[0]->type ==
NODE_BASIC_BLOCK)
        {
            current_block = flow_result.exits[0];
        }
        else
        {
            continue_current_block = false;
            current_block = createCFGNode(NODE_BASIC_BLOCK);
            addNode(cfg, current_block);

            for (int i = 0; i < flow_result.exit_count; i++)
            {
                CFGNode* exit_node = flow_result.exits[i];
                CFGEdge* exit_edge = flow_result.exitsEdges[i];
```

```
            exit_edge->to = current_block;
            addEdge(cfg, exit_edge);

            if (exit_edge->type == EDGE_TRUE)
                exit_node->nextConditional = current_block;

            else
                exit_node->nextDefault = current_block;
        }
    }

    current_block->statements = realloc(current_block->statements,
                                        (current_block->stmt_count + 1) *
                                        sizeof(OpNode*));
    current_block->statements[current_block->stmt_count++] = buildOpTree(node);

    if (continue_current_block)
        return flow_result;
    else
        {
        FlowResult flow_exits;

        flow_exits.exits = malloc(sizeof(CFGNode*));
        flow_exits.exits[0] = current_block;
        flow_exits.exitsEdges = malloc(sizeof(CFGEdge*));
        flow_exits.exitsEdges[0] = createCFGEdge(current_block, NULL, EDGE_CLASSIC);
        flow_exits.exit_count = 1;

        return flow_exits;
    }
  }
  else if (strcmp(node_text, "THEN") == 0 || strcmp(node_text, "ELSE") == 0
    || strcmp(node_text, "DO") == 0 || strcmp(node_text, "REPEATABLE_PART") == 0) {

    FlowResult new_flow_result = flow_result;
    for (ANTLR3_UINT32 i = 0; i < node->getChildCount(node); i++)
    {
        pANTLR3_BASE_TREE child = node->getChild(node, i);
        new_flow_result = processStatement(child, cfg, new_flow_result);
    }

    return new_flow_result;
  }
}
```

```
// Обработка IF statement
static FlowResult processIfStatement(pANTLR3_BASE_TREE if_node,
                        ControlFlowGraph* cfg,
                        FlowResult flow_entries)
{
    CFGNode* if_block = createCFGNode(NODE_IF);
    // Добавляем узел в граф
    addNode(cfg, if_block);

    for (int i = 0; i < flow_entries.exit_count; i++)
    {
        CFGNode* entry_node = flow_entries.exits[i];
        CFGEdge* entry_edge = flow_entries.exitsEdges[i];

        entry_edge->to = if_block;
        addEdge(cfg, entry_edge);

        if (entry_edge->type == EDGE_TRUE)
            entry_node->nextConditional = if_block;

        else
            entry_node->nextDefault = if_block;
    }

    // Находим condition, then и else части
    ANTLR3_UINT32 child_count = if_node->getChildCount(if_node);

    FlowResult end_of_then_block;
    FlowResult end_of_else_block;
    bool else_block_present = false;

    for (ANTLR3_UINT32 i = 0; i < child_count; i++)
    {
        pANTLR3_BASE_TREE child_node = if_node->getChild(if_node, i);

        if (strcmp(get_ast_node_text(child_node), "CONDITION") == 0)
        {
            pANTLR3_BASE_TREE condition_node = child_node;
            // Обработка блока statements
            if_block->statements = realloc(if_block->statements,
                                    (if_block->stmt_count + 1) *
                                    sizeof(OpNode*));
```

```
            if_block->statements[if_block->stmt_count++] = buildOpTree(condition_node);
        }

        else if (strcmp(get_ast_node_text(child_node), "THEN") == 0)
        {
            pANTLR3_BASE_TREE then_node = child_node;

            FlowResult if_block_flow;
            if_block_flow.exits = malloc(sizeof(CFGNode*));
            if_block_flow.exits[0] = if_block;
            if_block_flow.exitsEdges = malloc(sizeof(CFGEdge*));
            if_block_flow.exitsEdges[0] = createCFGEdge(if_block, NULL, EDGE_TRUE);
            if_block_flow.exit_count = 1;

            end_of_then_block = processStatement(then_node, cfg, if_block_flow);
        }
        else if (strcmp(get_ast_node_text(child_node), "ELSE") == 0)
        {
            pANTLR3_BASE_TREE else_node = child_node;

            else_block_present = true;

            FlowResult flow_before_else;
            flow_before_else.exits = malloc(sizeof(CFGNode*));
            flow_before_else.exits[0] = if_block;
            flow_before_else.exitsEdges = malloc(sizeof(CFGEdge*));
            flow_before_else.exitsEdges[0] = createCFGEdge(if_block, NULL, EDGE_FALSE);
            flow_before_else.exit_count = 1;

            end_of_else_block = processStatement(else_node, cfg, flow_before_else);
        }
    }

    FlowResult exit_flow_result;

    exit_flow_result.exit_count = 0;
    exit_flow_result.exits = malloc(0);
    exit_flow_result.exitsEdges = malloc(0);
    flowAppend(&exit_flow_result, end_of_then_block);

    if (else_block_present)
        flowAppend(&exit_flow_result, end_of_else_block);
    else
    {
```

```
        FlowResult if_block_flow;
        if_block_flow.exits = malloc(sizeof(CFGNode*));
        if_block_flow.exits[0] = if_block;
        if_block_flow.exitsEdges = malloc(sizeof(CFGEdge*));
        if_block_flow.exitsEdges[0] = createCFGEdge(if_block, NULL, EDGE_FALSE);
        if_block_flow.exit_count = 1;

        flowAppend(&exit_flow_result, if_block_flow);
    }

    return exit_flow_result;
}

// Обработка WHILE statement
static FlowResult processWhileStatement(pANTLR3_BASE_TREE while_node,
                                        ControlFlowGraph* cfg,
                                        FlowResult flow_entries) {

    // Создаем узлы для if
    CFGNode* while_block = createCFGNode(NODE_WHILE);
    // Добавляем узел в граф
    addNode(cfg, while_block);

    for (int i = 0; i < flow_entries.exit_count; i++)
    {
        CFGNode* exit_node = flow_entries.exits[i];
        CFGEdge* exit_edge = flow_entries.exitsEdges[i];

        exit_edge->to = while_block;
        addEdge(cfg, exit_edge);

        if (exit_edge->type == EDGE_TRUE)
            exit_node->nextConditional = while_block;

        else
            exit_node->nextDefault = while_block;
    }

    // Находим condition, then и else части
    ANTLR3_UINT32 child_count = while_node->getChildCount(while_node);

    FlowResult end_of_do_block;

    for (ANTLR3_UINT32 i = 0; i < child_count; i++)
```

```
    {
        pANTLR3_BASE_TREE child_node = while_node->getChild(while_node, i);

        if (strcmp(get_ast_node_text(child_node), "CONDITION") == 0)
        {
            pANTLR3_BASE_TREE condition_node = child_node;
            // Обработка блока statements
            while_block->statements = realloc(while_block->statements,
                                              (while_block->stmt_count + 1) *
                                              sizeof(OpNode*));
            while_block->statements[while_block->stmt_count++] =
buildOpTree(condition_node);
        }

        else if (strcmp(get_ast_node_text(child_node), "DO") == 0)
        {
            pANTLR3_BASE_TREE do_node = child_node;
            // Обрабатываем then часть
            FlowResult while_block_flow;
            while_block_flow.exits = malloc(sizeof(CFGNode*));
            while_block_flow.exits[0] = while_block;
            while_block_flow.exitsEdges = malloc(sizeof(CFGEdge*));
            while_block_flow.exitsEdges[0] = createCFGEdge(while_block, NULL,
EDGE_TRUE);
            while_block_flow.exit_count = 1;

            end_of_do_block = processStatement(do_node, cfg, while_block_flow);

            for (int k = 0; k < end_of_do_block.exit_count; k++)
            {
                CFGNode* exit_node = end_of_do_block.exits[k];
                CFGEdge* exit_edge = end_of_do_block.exitsEdges[k];

                exit_edge->to = while_block;
                addEdge(cfg, exit_edge);

                if (exit_edge->type == EDGE_TRUE)
                    exit_node->nextConditional = while_block;

                else
                    exit_node->nextDefault = while_block;
            }
        }
    }
```

```
    FlowResult flow_exits;

    flow_exits.exits = malloc(sizeof(CFGNode*));
    flow_exits.exits[0] = while_block;
    flow_exits.exitsEdges = malloc(sizeof(CFGEdge*));
    flow_exits.exitsEdges[0] = createCFGEdge(while_block, NULL, EDGE_FALSE);
    flow_exits.exit_count = 1;

    return flow_exits;
}

// Обработка REPEAT statement
static FlowResult processRepeatStatement(pANTLR3_BASE_TREE repeat_node,
                                         ControlFlowGraph* cfg,
                                         FlowResult flow_entries) {

    // Создаем узлы для if
    CFGNode* repeatable_part_block = createCFGNode(NODE_BASIC_BLOCK);
    // Добавляем узел в граф
    addNode(cfg, repeatable_part_block);

    for (int i = 0; i < flow_entries.exit_count; i++)
    {
        CFGNode* exit_node = flow_entries.exits[i];
        CFGEdge* exit_edge = flow_entries.exitsEdges[i];

        exit_edge->to = repeatable_part_block;
        addEdge(cfg, exit_edge);

        if (exit_edge->type == EDGE_TRUE)
            exit_node->nextConditional = repeatable_part_block;

        else
            exit_node->nextDefault = repeatable_part_block;
    }

    // Находим condition, then и else части
    ANTLR3_UINT32 child_count = repeat_node->getChildCount(repeat_node);

    FlowResult end_of_repeatable_part_flow;

    for (ANTLR3_UINT32 i = 0; i < child_count; i++)
    {
```

```
        pANTLR3_BASE_TREE child_node = repeat_node->getChild(repeat_node, i);

        if (strcmp(get_ast_node_text(child_node), "REPEATABLE_PART") == 0)
        {
            pANTLR3_BASE_TREE repeatable_part_node = child_node;

            FlowResult flow_repeatable_part;

            flow_repeatable_part.exits = malloc(sizeof(CFGNode*));
            flow_repeatable_part.exits[0] = repeatable_part_block;
            flow_repeatable_part.exitsEdges = malloc(sizeof(CFGEdge*));
            flow_repeatable_part.exitsEdges[0] = createCFGEdge(repeatable_part_block, NULL,
EDGE_CLASSIC);
            flow_repeatable_part.exit_count = 1;

            end_of_repeatable_part_flow = processStatement(repeatable_part_node, cfg,
flow_repeatable_part);
        }
    }

    FlowResult flow_exits;

    for (ANTLR3_UINT32 i = 0; i < child_count; i++)
    {
        pANTLR3_BASE_TREE child_node = repeat_node->getChild(repeat_node, i);

        if (strcmp(get_ast_node_text(child_node), "UNTIL") == 0)
        {
            pANTLR3_BASE_TREE condition_node = child_node;
            // Обработка блока statements

            CFGNode* until_block = createCFGNode(NODE_REPEAT_CONDITION);
            addNode(cfg, until_block);

            until_block->statements = realloc(until_block->statements,
                                              (until_block->stmt_count + 1) *
                                              sizeof(OpNode*));
            until_block->statements[until_block->stmt_count++] =
buildOpTree(condition_node);

            for (int k = 0; k < end_of_repeatable_part_flow.exit_count; k++)
            {
                CFGNode* exit_node = end_of_repeatable_part_flow.exits[k];
                CFGEdge* exit_edge = end_of_repeatable_part_flow.exitsEdges[k];
```

```
            exit_edge->to = until_block;
            addEdge(cfg, exit_edge);

            if (exit_edge->type == EDGE_TRUE)
                exit_node->nextConditional = until_block;
            else
                exit_node->nextDefault = until_block;
        }

        CFGEdge* until_to_repeatable_part = createCFGEdge(until_block,
repeatable_part_block, EDGE_TRUE);

        addEdge(cfg, until_to_repeatable_part);

        if (until_to_repeatable_part->type == EDGE_TRUE)
            until_block->nextConditional = repeatable_part_block;
        else
            until_block->nextDefault = repeatable_part_block;

        flow_exits.exits = malloc(sizeof(CFGNode*));
        flow_exits.exits[0] = until_block;
        flow_exits.exitsEdges = malloc(sizeof(CFGEdge*));
        flow_exits.exitsEdges[0] = createCFGEdge(until_block, NULL, EDGE_FALSE);
        flow_exits.exit_count = 1;
    }
  }

  return flow_exits;
}
```

3 Тестирование модуля на работоспособность

Для тестирования модуля на работоспособность разработано более 5 тест кейсов, каждый из которых проверяет построение графа потока управления для методов с различными синтаксическими конструкциями, однако, ниже приведён один из самых сложных примеров.

Входной файл (calc.txt):

```
method calc(x : int, op : char, y : int)
var result : int;
```

```
begin
   if op == '+' then result := x + y;
   if op == '-' then result := x - y;
   if op == '*' then result := x * y;
   if op == '/' then result := x / y;
   write(result);
end;

method fib(n : int) : int
var result : int;
begin
   if n <= 2 then
      result := 1;
   else
      result := fib(n - 2) + fib(n - 1);

   result;
end;

method main()
var x, y : int;
op : char;
begin
   x := read();
   op := read();
   if op == 'f' then
      write(fib(x));
   else begin
      y := read();
      calc(x, op, y);
   end;
end;


method read();
method write(num : int);
```

По входному файлу созданы calc.calc.dot, calc.fib.dot, calc.main.dot, calc.read.dot, calc.write.dot, calc.callgraph.dot. Ниже приведены картинки получившихся графов потока управления, а также графа вызова.

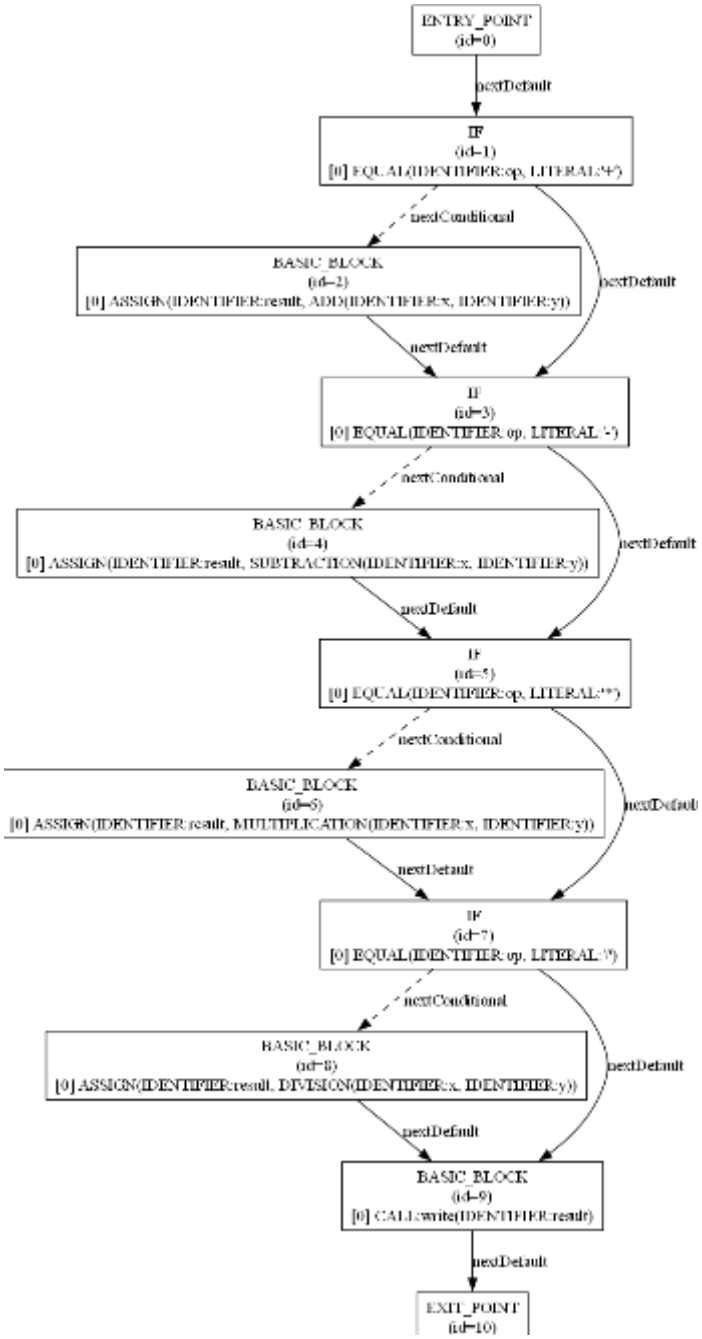

Рисунок 1 – Граф потока управления для входного метода calc

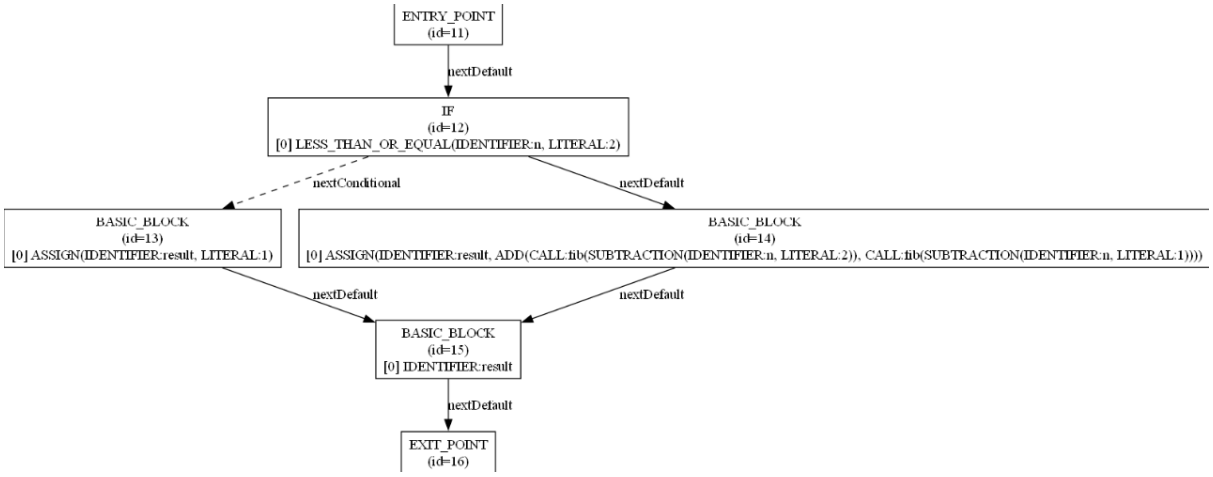

Рисунок 2 – Граф потока управления для входного метода fib

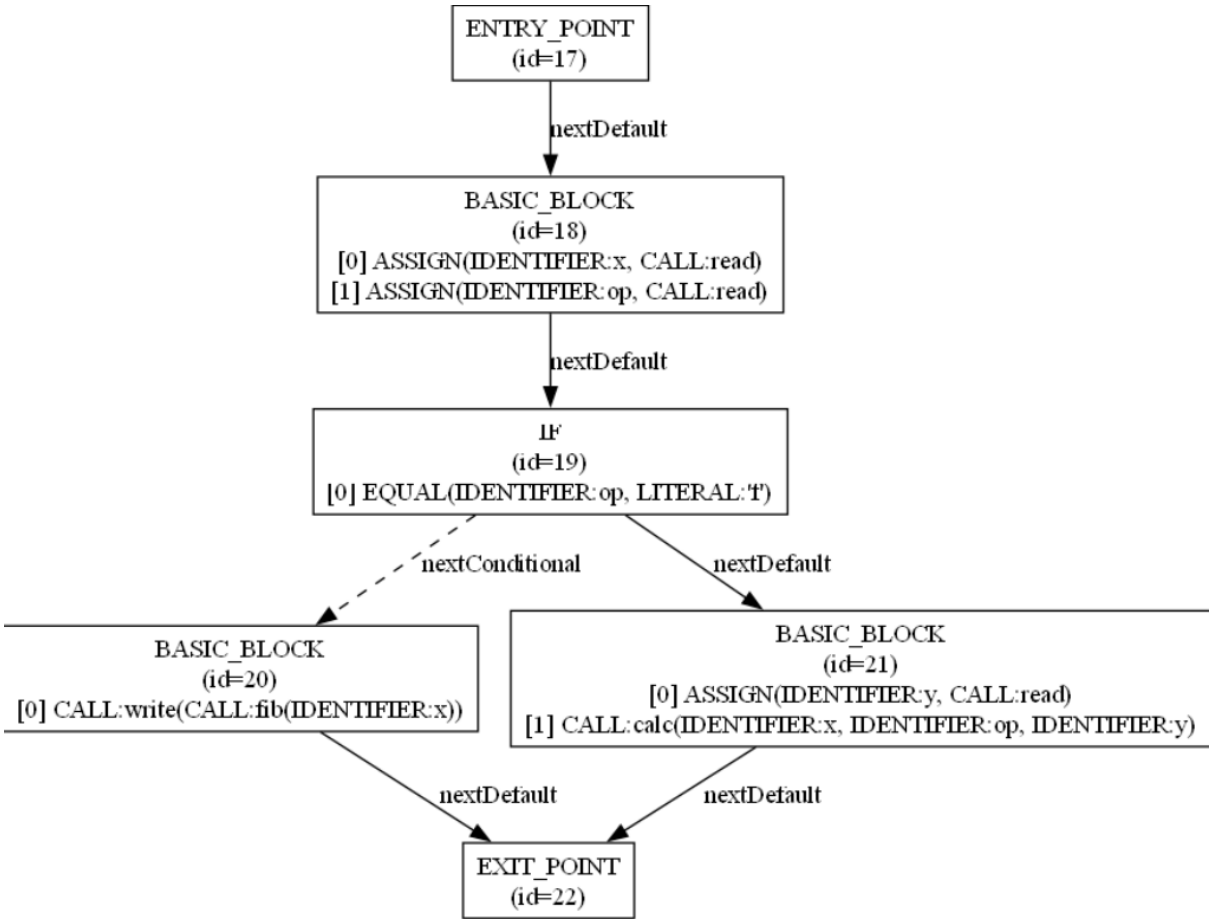

Рисунок 3 – Граф потока управления для входного метода main

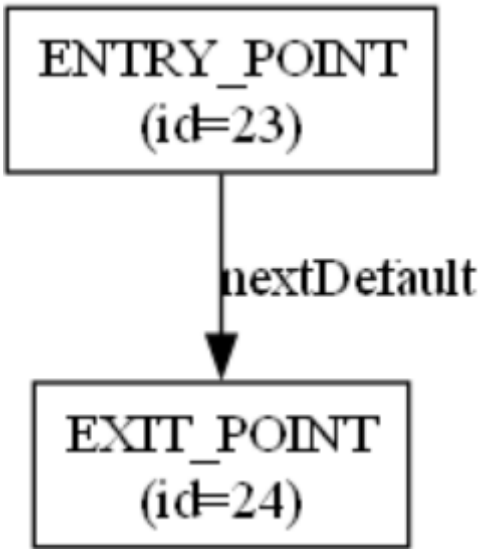

Рисунок 4 – Граф потока управления для входного метода read

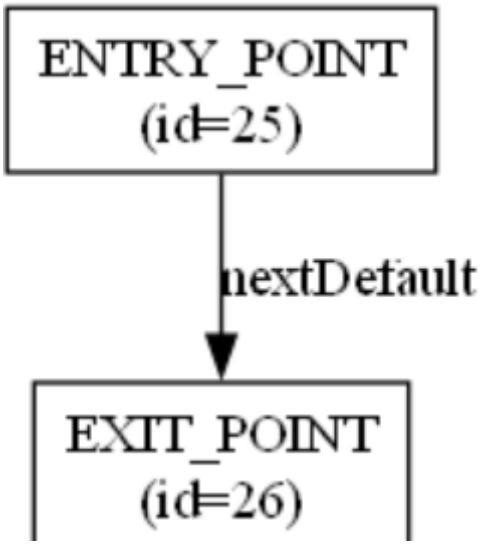

Рисунок 5 – Граф потока управления для входного метода write

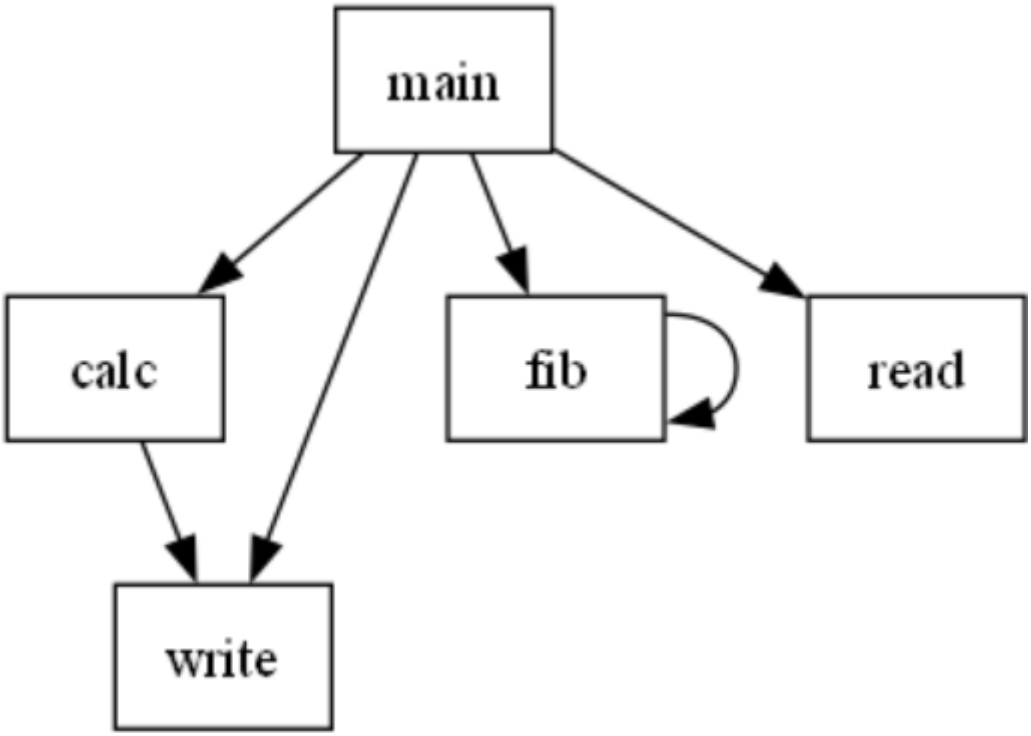

Рисунок 6 – Граф вызовов

Вывод:

В ходе практической работы разработана программа для построения графов потока управления для методов внутри входных файлов с выводом графа вызовов обращения методов друг к другу. В результате полученная программа анализирует входной набор файлов и выводит .dot представления для всех методов и .dot файл для графа вызовов.